

CP372 (Section C) – Assignment 01

RFC: Bulletin Board Client–Server Protocol

Document Status: Informational

Intended Audience: CP372 Assignment 01

Authors: Viraj Gill, Dante Rendell

Version: 1.3

Date: 30 January 2026

1. Introduction

1.1 Purpose of This Document

This RFC document provides information for users/clients by formally specifying the application-layer protocol used for the Bulletin Board System. It outlines rules for message formatting, what each command does (semantics), and how the system should handle errors on both the client and server ends.

1.2 Scope

This document defines the communication standards between the client and server for the Bulletin Board System. It focuses on the application-layer protocol and its syntax, semantics, and error handling for all commands. Aspects such as the GUI design, aesthetics, and the technical details of the transport layer are outside of the scope of this RFC document. Additionally, persistence is not a goal for this system, meaning that any data after the server shuts down is also out of scope.

1.3 Design Goals

The program prioritizes simplicity, so a plain-text, and line-based communication style is used to make every message easier to parse, test, and debug. This approach supports a predictable request-response pattern and a basic, intuitive GUI for the end-user.

Stability is a primary concern for this system; error handling is included for every potential case to ensure the server keeps running smoothly. Defining structured error codes, such as `INVALID_FORMAT` or `OUT_OF_BOUNDS`, allows the system to provide immediate and helpful feedback.

The server uses multithreading and synchronization in order to handle multiple client connections. These methods will keep the server data safe from corruption. Operations like SHAKE, POST, and CLEAR are fully atomic. This means when a client uses the refresh button to trigger a GET command, a consistent and up-to-date board state is always returned. Glitches or partial updates won't happen during concurrent edits because these requests are handled one at a time.

2. System Overview

2.1 Architectural Model

The system follows a centralized client-server architecture, which is a common setup for distributed applications (e.g., Netflix, Wikipedia). The server acts as the primary authority; it manages the entire bulletin board system while multiple concurrent clients connect over TCP stream sockets to interact with notes, pins, and the board state. Because the server handles all board-related data in its memory, it must ensure that all connected clients see a consistent and accurate view of the board. Communication between the server and its clients is based on a request-response interaction pattern.

2.2 Connection Lifecycle

Server Startup:

- The server is started with `java Server <port> <board_width> <board_height> <note_width> <note_height> <color1> ... <colorN>`.
- A server socket is opened with the specified port.
- The server will be configured with the rest of the specified parameters (board dimensions, note dimensions, and valid note colors).

Client Connection:

- The client uses a simple server connection in which the end user is prompted to input the server's IP address and port (via a GUI interface).
- The end user then confirms their input by clicking on the Connect button.
- The client will attempt to use the default IP address (localhost) and the default port if not specified by the user.

Handshake Phase:

- The server accepts an incoming connection from the client and spawns a new thread to handle the client.

- The server sends over initial information about itself (board dimensions, note dimensions, valid colors).
- Once the client receives the initial information, it is free to send commands until the connection is terminated.

Active Command Phase:

- The client is free to send commands at any time while connected to the server.
- Each client is handled on its own thread, so the server can service many commands synchronously.
- The server will return the status of the command, followed by content from the server if the command requested it.

Disconnection:

- The client can send a `DISCONNECT` command at any time, in which case the connection between the client and server will be terminated.

3. Terminology and Definitions

Board: The bulletin board is a fixed rectangular surface with dimensions that are determined at server start-up.

Note: A rectangular object defined by its top-left coordinates, color, and message. All notes have a determined width and height at server start-up.

Pin: A single (x, y) coordinate that, when placed inside a note, changes the status of that note into a pinned note.

Pinned Note: Any note containing at least one active pin is considered as “pinned”.

Client: Refers to the GUI program the end-user interacts with. Connects to the server to send commands.

Server: The central Java application that manages the bulletin board, the note and pin data, and client connections.

Atomic Operation: Operations such as `POST`, `SHAKE`, and `CLEAR` that are uninterruptible must finish execution before handling the next request.

Complete Overlap: Two notes that share the same x and y coordinates completely overlap each other.

Color: Refers to the color property of any given note. Colors are specified at server startup and must be a JavaFX color.

4. Data Model

4.1 Coordinate System

Origin:

- The upper-left corner of the bulletin board (defined as 0, 0).

Axis Directions:

- The x-axis starts at 0 and increases in value horizontally to the right.
- The y-axis starts at 0 and increases in value going down vertically.

Valid Coordinate Range:

- Points must stay within the specific board width and height defined at server startup.

4.2 Board Properties

Width and Height:

- The width and height of the bulletin board are specified in the server startup command/
- Specifies the bottom right corner of the board.
- Uses positive integer values only.

Boundary Rules:

- Notes must be within the board boundaries.
- Cannot exceed (board_width - note_width, board_height - note_height).

4.3 Note Properties

Position:

- This is the x and y coordinate for the top-left corner of the note.

Dimensions:

- The width and height of every note are fixed by the server at startup.
- All notes are the same size.

Color:

- Every note has a color property that must be chosen from the server's valid list of colors.

Content:

- The message string that the note displays for the client users to read.

Pin Status Derivation:

- Notes start as unpinned when they are first posted.
- A note is marked as pinned if it has at least one active pin currently associated with it.
- Unpinned notes will be removed with the SHAKE command

4.4 Pin Properties

Pin Coordinates:

- Defined by a single x and y coordinate point on the board.
- Pins cannot share x and y coordinates.

Relationship to Notes:

- Pin coordinates must land inside an existing note's rectangular area (cannot be along the edges of a note).
- Successfully adding a pin to the board will increase a note's pin amount by 1.
- A pin can fall inside the rectangular area of multiple notes that partially overlap each other (will also change the status of all the unpinned notes for that area).

Multiple Pins per Note:

- A single note can have more than one pin stuck in it at the same time.

Pin Removal Semantics:

- Removing a pin from a note will decrease # of pins associated with that note.
- Removing a pin from a note won't change its pinned status unless it was the last pin.

5. Protocol Overview

5.1 Transport Protocol

The system relies on TCP stream sockets to ensure a reliable and ordered delivery of data between the client and server. Messages are plain-text, follow a line-oriented format, and must terminate with a single newline/linefeed character. This setup ensures data is simple and easy to parse.

5.2 Message Exchange Pattern

The client-server architecture is built around a request-response pattern where the client always initiates the interaction. Once a client sends a command, the server processes it and sends exactly one response back. The server will never send data unprompted without an initial client request.

6. Message Format Specification

6.1 General Message Syntax

This protocol uses a text-based format where:

- Fields are separated by a single ASCII space character (i.e. SP, 0x20).
- No extra whitespace should be present in a message.

- Each command must end with a newline/linefeed character (i.e. LF, 0x0A).

6.2 Formal Grammar (ABNF)

6.2.1 POST

```
command = "POST" SP x SP y SP color SP message LF
```

```
x = integer
```

```
y = integer
```

```
integer = 1*(DIGIT)
```

```
color = 1*(ALPHA)
```

```
message = 1*(VCHAR | SP)
```

```
response = success | error
```

```
success = "OK NOTE_POSTED" LF
```

```
error = "ERROR" SP error-code LF
```

```
error-code = "INVALID_FORMAT" |  
            "OUT_OF_BOUNDS" |  
            "COLOR_NOT_SUPPORTED" |  
            "COMPLETE_OVERLAP"
```

6.2.2 GET

```
command = "GET" type LF
```

```
type = SP "PINS" | *(SP parameter)
```

```
parameter = color-param | contains-param | refersTo-param
```

```
color-param = "color=" color
```

```
contains-param = "contains=" x SP y
```

```
refersTo-param = "refersTo=" substring
```

```
x = integer
```

```
y = integer
```

```
integer = 1*(DIGIT)
```

```
color = 1*(ALPHA)
```

```
substring = 1*(VCHAR)
```

```
response = success | error
```

```
success = "OK" SP success-code SP integer returned-lines LF
```

```
success-code = "PIN" | "NOTE"
```

```
returned-lines = *(LF success-code obj-data)
obj-data = pin-data | note-data
pin-data = integer SP integer
note-data = integer SP integer SP color SP 1*(VCHAR | SP)
error = "ERROR" SP error-code LF
error-code = "INVALID_FORMAT" |
             "OUT_OF_BOUNDS" |
             "COLOR_NOT_SUPPORTED" |
             "NO_NOTE_AT_COORDINATE"
```

6.2.3 PIN

```
command = "PIN" SP x SP y LF
x = integer
y = integer
integer = 1*(DIGIT)

response = success | error
success = "OK PIN_ADDED" LF
error = "ERROR" SP error-code LF
error-code = "INVALID_FORMAT" |
             "OUT_OF_BOUNDS" |
             "PIN_ALREADY_EXISTS" |
             "NO_NOTE_AT_COORDINATE"
```

6.2.4 UNPIN

```
command = "UNPIN" SP x SP y LF
x = integer
y = integer
integer = 1*(DIGIT)

response = success | error
success = "OK PIN_REMOVED" LF
error = "ERROR" SP error-code LF
error-code = "INVALID_FORMAT" |
             "OUT_OF_BOUNDS" |
             "PIN_NOT_FOUND"
```

6.2.5 SHAKE

```
command = "SHAKE" LF
```

```
response = success | error
success = "OK SHAKE_COMPLETE" LF
error = "ERROR EMPTY_BOARD" LF
```

6.2.6 CLEAR

```
command = "CLEAR" LF
```

```
response = success | error
success = "OK CLEARED" LF
error = "ERROR EMPTY_BOARD" LF
```

6.2.7 DISCONNECT

```
command = "DISCONNECT" LF
response = "OK DISCONNECTED" LF
```

7. Client-to-Server Commands

7.1 POST Command

Post a note to the board with the given position, color, and message.

7.1.1 Syntax

```
POST <x> <y> <color> <message...>
```

7.1.2 Parameters

<x>

- integer
- The top left x coordinate of a note.

<y>

- integer
- The top left y coordinate of a note.

<color>

- String
- Must be one of the colors defined on server start.

<message>

- String
- The string data to be displayed on the note.

7.1.3 Preconditions

- `<color>` must be one of the valid colors defined in the server. This list is sent to the client upon first connection.
- `<x>` `<y>` must be valid coordinates on the board.

7.1.4 Server-side Validation

- Verify `<x>` and `<y>` are positive integers within the bounds of the board.
- Verify `<color>` is one of the valid colors specified upon server startup.

7.1.5 Success Response

`"OK NOTE_POSTED"`

- Note was successfully posted to the board.

7.1.6 Error Responses

`"ERROR_INVALID_FORMAT"`

- The command was formatted incorrectly.

`"ERROR_OUT_OF_BOUNDS"`

- The coordinates provided were outside the boundaries of the board.

`"ERROR_COLOR_NOT_SUPPORTED"`

- The color provided was not in the list of valid colors.

`"ERROR_COMPLETE_OVERLAP"`

- The note completely overlaps another note already on the board (has the same x and y coordinates).

7.2 GET Command

Can fetch all the pins currently on the board OR get all notes with the provided filters. If no filters are applied, all note data is implied.

7.2.1 Syntax

`GET PINS`

OR

`GET color=<color> contains=<x> <y> refersTo=<substring>`

7.2.2 Parameters

If `GET PINS` is used, no other parameter is accepted. Otherwise, the following parameters are allowed. If no parameter is present, all note data is implied.

`<x>`

- integer
- Filter notes containing this x-axis coordinate.

`<y>`

- integer
- Filter notes containing this y-axis coordinate.

<color>

- String
- Must be one of the colors defined on server start.
- Notes with this color will be returned.

<substring>

- String
- A substring used to identify the message contents of an existing note.
- Only notes that contain this substring will be returned.

7.2.3 Preconditions

- <x> <y> must be valid coordinates on the board.
- <color> must be one of the valid colors defined in the server. This list is sent to the client upon first connection.

7.2.4 Server-side Validation

- Verify <x> and <y> are positive integers within the bounds of the board.
- Verify <x> and <y> are coordinates within an existing note for the filter-based GET command.
- Verify <color> is one of the valid colors specified upon server startup.

7.2.5 Success Response

GET PINS:

"OK PIN <# of objects returned>"

"PIN <x> <y>"

- This response shows that pins were returned and the number of pins returned.
- It will be followed by <# of objects returned> lines containing the coordinates for the pins.

GET color=<color> contains=<x> <y> refersTo=<substring>:

"OK NOTE <# of objects returned>"

"NOTE <x> <y> <color> <message> <pinned>"

- This response shows that notes were returned and the number of notes returned.
- It will be followed by <# of objects returned> lines containing the coordinates for the notes, the color, the message, and if the note is currently pinned.

7.2.6 Error Responses

"ERROR INVALID_FORMAT"

- The command was formatted incorrectly.
"ERROR_OUT_OF_BOUNDS"
- The coordinates provided were outside the boundaries of the board.
"ERROR_COLOR_NOT_SUPPORTED"
- The color provided was not in the list of valid colors.

7.3 PIN Command

Places a pin on the board at the provided coordinates. If there are multiple notes under the pin, they will all become pinned. A pin cannot be placed on another pin.

7.3.1 Syntax

PIN <x> <y>

7.3.2 Parameters

<x>

- integer
- The x coordinate of a pin.

<y>

- integer
- The y coordinate of a pin.

7.3.3 Preconditions

- <x> and <y> must be valid coordinates on the board.

7.3.4 Server-side Validation

- Verify <x> and <y> are positive integers within the bounds of the board.
- Verify <x> and <y> are coordinates within an existing note.
- Verify the pin doesn't already exist at the coordinates.

7.3.5 Success Response

"OK_PIN_ADDED"

- Pin was successfully added to the board.

7.3.6 Error Responses

"ERROR_INVALID_FORMAT"

- The command was formatted incorrectly.

"ERROR_OUT_OF_BOUNDS"

- The coordinates provided were outside the board boundaries.

"ERROR_PIN_ALREADY_EXISTS"

- There is already a pin at the provided coordinates.

7.4 UNPIN Command

A pin at the provided coordinates will be removed from the board. If there is no pin at the provided coordinates, an error will be returned. A note will not be marked as unpinned until all pins are removed from its area.

7.4.1 Syntax

UNPIN <x> <y>

7.4.2 Parameters

<x>

- integer
- The x coordinate of a pin.

<y>

- integer
- The y coordinate of a pin.

7.4.3 Preconditions

- <x> and <y> must be valid coordinates on the board.
- <x> and <y> must correspond to the coordinates of an existing pin.

7.4.4 Server-side Validation

- Verify <x> and <y> are coordinates that belong to an existing pin.

7.4.5 Success Response

"OK PIN_REMOVED"

- The pin at the provided was successfully removed.

7.4.6 Error Responses

"ERROR PIN_NOT_FOUND"

- There was no pin found at the coordinates provided.

"ERROR INVALID_FORMAT"

- The command was formatted incorrectly.

"ERROR OUT_OF_BOUNDS"

- The coordinates provided were outside the boundaries of the board.

7.5 SHAKE Command

Removes all unpinned notes from the board. Pinned notes remain untouched. This is done atomically, so no other commands will be able to access the board and pin state when a shake command is received.

7.5.1 Syntax

`"SHAKE"`

7.5.2 Parameters

- No parameters should be passed with SHAKE.

7.5.3 Preconditions

- Needs to have unpinned notes on the board to have any effect. If no unpinned notes are found, this simply has no effect.

7.5.4 Server-side Validation

- Make sure no extra parameters are provided with the command.
- Make sure only unpinned notes are removed from the board.

7.5.5 Success Response

`"OK SHAKE_COMPLETE"`

- Successfully removed all unpinned notes.

7.5.6 Error Responses

`"ERROR_INVALID_FORMAT"`

- The command was formatted incorrectly.

`"ERROR_EMPTY_BOARD"`

- The board does not have any notes.

7.6 CLEAR Command

Removes all notes and pins from the board, regardless of the pinned state of the note. This is done atomically, so no other commands will be able to access the board and pin state when a clear command is received.

7.6.1 Syntax

`"CLEAR"`

7.6.2 Parameters

- No parameters should be passed with clear.

7.6.3 Preconditions

- The board must have at least one note or pin.

7.6.4 Server-side Validation

- Verify there is at least one note or pin to be cleared.

7.6.5 Success Response

`"OK CLEARED"`

- All pins and notes have been successfully cleared.

7.6.6 Error Responses

`"ERROR EMPTY_BOARD"`

- The board does not have any notes or pins.

7.7 DISCONNECT Command

Disconnect the client from the server. The server will disconnect the client and close the thread that was handling it.

7.7.1 Syntax

`"DISCONNECT"`

7.7.2 Parameters

- No parameters should be passed with disconnect.

7.7.3 Preconditions

- The client must be connected to the server.

7.7.4 Server-side Validation

- No validation necessary.

7.7.5 Success Response

`"OK DISCONNECTED"`

- Successfully disconnected from the server.

7.7.6 Error Responses

- No error responses. The client will disconnect no matter what.

8. Server-to-Client Responses

8.1 Success Responses

A general success response is "OK", followed by a space, then the success code. All success responses are in plain text and are human-readable.

```
general-success = "OK" SP 1*(ALPHA | "_") LF
```

Data-bearing responses have a longer success message. It starts with a general success response, but it adds an integer to the end indicating the number of objects it is returning. The successful response will be followed by the data that was requested.

```
data-bearing-success = "OK" SP success-code SP integer data LF
success-code = 1*(ALPHA)
data = *(LF obj)
obj = 1*(VCHAR | SP)
```

8.2 Error Responses

Every error message begins with "ERROR", followed by a space, then the error code. All error codes are in plain text and are human-readable. An example of an error code that could appear is: "ERROR_INVALID_FORMAT".

```
error = "ERROR" SP 1*(ALPHA | "_") LF
```

9. Error Handling

9.1 Error Classification

```
INVALID_FORMAT
OUT_OF_BOUNDS
COLOR_NOT_SUPPORTED
COMPLETE_OVERLAP
PIN_NOT_FOUND
EMPTY_BOARD
```

NO_NOTE_AT_COORDINATE
PIN_ALREADY_EXISTS

9.2 Client-Side Responsibilities

- Validate all user input before a request is transmitted to the server.
 - Check coordinates against board width and height provided during the handshake phase to ensure they are within the boundaries.
 - Colors must be from the list of valid JavaFX colors received from the server.
 - Message fields should not be empty or contain illegal characters that might break the line-oriented format.
- Handle situations where the server becomes unreachable or the connection is lost during a command.
- Provide clear feedback on the GUI if an input is invalid.

9.3 Server-Side Responsibilities

- Make sure all command parameters follow the defined specification.
 - All coordinates must be inside the board.
 - Notes must not completely overlap each other (cannot share the same coordinates).
 - All colors must be valid/defined on server start.
- Ensure multiple clients' requests can be handled concurrently.
- Use thread-safe functions for handling mutable data.
- For reads of mutable data, return a copy of the data instead of a reference.

10. Concurrency and Synchronization

10.1 Multithreading Model

The server runs on a Thread-per-client model. When the server accepts an incoming client, it spawns a new thread to handle all interactions between the server and the accepted client.

10.2 Shared Data Protection

The server stores all information about the server in a shared `ServerCtx` object. `ServerCtx` consists of a `ServerConfig` and `ServerState` object. `ServerConfig` holds immutable information about the server that can be accessed at any time with no synchronization or locking necessary. `ServerState` contains mutable information about the server that must be accessed through synchronous methods that are executed atomically to prevent race conditions.

10.3 Consistency Guarantees

Clients receive a consistent version of the board because every command is processed against the most recent state. Synchronization is used on `ServerState` to ensure all mutable data changes are atomic.

11. Border and Failure Cases

An empty board is a valid state for the server to have. It simply means there are no notes posted. `SHAKE` and `CLEAR` throw an “EMPTY_BOARD” error if they are issued on an empty board. If a `GET` command is issued in this state, it is not an error. The `GET` will simply return an “OK” with 0 objects.

If a client disconnects mid-request, the request will finish being serviced if it is possible. The server will then exit from the thread that was handling the connected client.

If there are no clients connected to the server, a 10-minute timer starts counting down. If no new clients connect during this time, the server will shut down. If a client does connect during this countdown, the timer is paused and reset to 10 minutes. The server shutdown simply closes the server socket, and the program ends.

12. Security Considerations

Security for this system is based on the idea that it runs in a trusted environment, so there is no formal encryption or authentication. The main defence is strict input validation on the server’s end. This ensures that no one can send data that could corrupt the memory or crash the threads. In terms of Denial-of-service, the system has no specific protections against flood attacks. However, the server’s multithreaded nature can handle many clients at once without it slowing down too much.

13. Division of Responsibilities

Dante is the lead for the server and will handle the multithreading, synchronization, and the memory-based note data. Viraj is the lead for the client GUI, and he is responsible for the visual interface and validating user input before transmission.

The protocol design, additional Java files, and RFC development are shared responsibilities. GitHub is used for version control to coordinate work. Both members are responsible for ensuring the system is bug-free and reviewing each other's code for a smooth, working system. Work is not exclusive, so we will have to look out for each other to maintain quality and functionality.

14. Example Message Exchanges (Appendix A)

Valid POST Example:

Client -> Server

```
POST 5 10 white First Example message
```

Server -> Client

```
OK NOTE_POSTED
```

Valid GET Example:

Client -> Server

```
GET color=white contains=6 8 refersTo=Example
```

Server -> Client

```
OK NOTE 1
```

```
NOTE 5 10 white First Example message PINNED=false
```

Error Response Example:

Client -> Server

```
UNPIN 1 2
```

Server -> Client

```
ERROR PIN_NOT_FOUND
```

Concurrent Scenario Example:

Client #1 -> Server (Concurrent, Thread #1)

PIN 1 2

Client #2 -> Server (Concurrent, Thread #2)

CLEAR

Server -> Client #1

OK PIN_ADDED

Server -> Client #2

OK CLEARED

15. Revision History (Appendix B)

1.0

23 January 2026

Preliminary draft of the RFC document. Sections were copied over from the “Model RFC Outline” on the D2L course website.

1.1

25 January 2026

Initial work was done on sections 1, 3, and 14.

1.2

29 January 2026

Filled in sections 2, 4, 6, 7, 9, and 10.

1.3

30 January 2026

Final draft of the RFC document.